GH-300

Tools & Agents

in GitHub Copilot

From built-in tools to autonomous agents — plan, code, review, ship.

Built-in | MCP | Extensions | Plan · Ask · Agent · Custom | Agent Host · Cloud

Sandeep Soni · Deccansoft Software Services Pvt Ltd

Deck 9 · GH-300 Certified GitHub Copilot Expert Training

Agenda

1. What is an agent — and what is the agent loop
2. Three tool types: Built-in · MCP · Extensions
3. Using tools in prompts; tool sets and limits
4. Four local modes: Agent · Plan · Ask · Custom
5. Five harnesses: Local · Copilot · Claude · Codex · Cloud
6. Permissions: Default · Assisted · Bypass, plus Autopilot mode
7. Folder vs Worktree isolation
8. Local vs Agent Host — when each one wins
9. Agent handoff workflow: Plan → Copilot → Cloud

CONCEPTS

What is an Agent?

An AI assistant that works autonomously to complete a coding task — given a goal, it plans, picks tools, executes, observes results, and iterates until done.

Autonomous execution

Takes a high-level goal and breaks it into steps without constant prompting.

Multi-file editing

Identifies relevant files across the project and edits them in coordination.

Tool invocation

Uses built-in, MCP, and extension tools as needed — chosen by the model, not pre-scripted.

Self-correction

Monitors output (compiler, tests, lint), detects failure, replans, retries.

Decision making

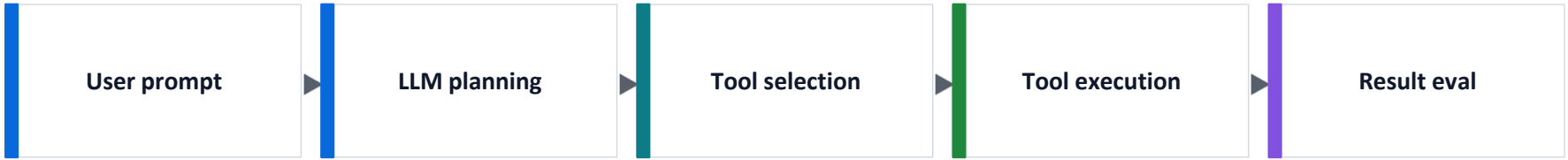
Chooses approach based on context — Plan vs Agent vs Edit, model vs tool, etc.

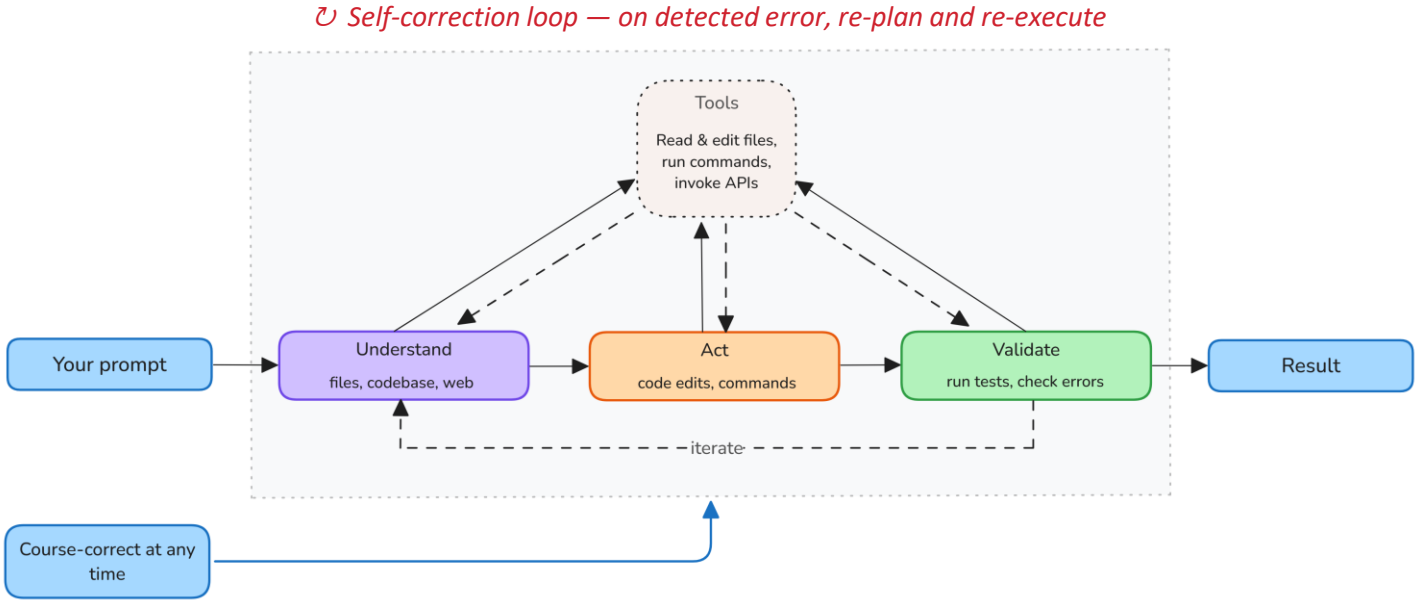
Memory (public preview)

Repo facts and user preferences.
28-day expiry.
Used by cloud agent, code review, CLI.

Example prompt: "Add login functionality to my app" — the agent decides which files to read, which to write, which packages to install, and which tests to run.

What is an Agent Loop?





The Agent Loop — What is sent to LLM

What the loop sends to the LLM on each turn

- System prompt — agent role + instructions for this mode (Agent / Plan / Ask / Custom).
- Conversation history — earlier turns, auto-summarised when the window fills.
- Tool definitions — built-in + MCP + extensions currently enabled.
- Machine context — OS, workspace info, active editor, selection.
- Custom instructions — .github/copilot-instructions.md + AGENTS.md + path-scoped *.instructions.md.
- Previous tool results + errors — so the agent can correct course.

Agent harness — the runtime that runs the agent loop

1

Agent harness

Coordinates the agent loop, calls tools and applies code changes.

2

Execution environment

Where tool calls and code changes actually run: your machine, a remote machine, or cloud infrastructure.

3

Agent role

The instructions, tools and behaviour for a task: Agent, Plan, Ask, or a custom agent you define.

4

Language model

Supplies the reasoning and the responses. It can run in a different location from the harness.

Session Target is the VS Code control that picks the harness and where it runs. The harness passes your prompt and context to the model, runs the tool calls it requests, and keeps looping until the task is done.

Session targets — where each harness runs

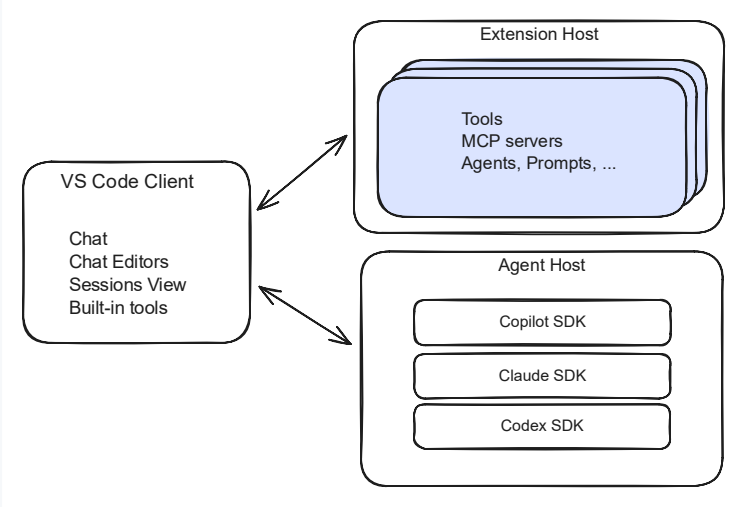
	Local	Agent Host Copilot · Claude · Codex	Cloud
Where	VS Code extension host	Separate process on your machine	GitHub's infrastructure
How	Interactive, real-time	Background, non-blocking	Async, fully autonomous
Tool access	All — built-in + MCP + ext.	Built-in + MCP, harness SDK	Cloud tools + GitHub Actions
Parallel?	No — one active	Yes — many sessions	Yes
Best for	Interactive coding, debug	Parallel work in worktrees	Issue → PR, research, branch work

Supported harnesses — the engines behind a session

Harness	Engine / SDK	Runs on	Tools and models
Local	Built-in VS Code harness	Extension host	VS Code + extension tools, MCP servers, VS Code models
Copilot	Copilot SDK	Agent Host	Behaviour matches Copilot CLI and the Copilot app
Claude	Anthropic Claude Agent SDK	Agent Host	Anthropic or Copilot model groups
Codex	OpenAI Codex	Agent Host	Local sessions need the Codex extension
Cloud	Provider you choose after picking Cloud	GitHub infrastructure	Cloud tools and models; result is a pull request

The Agent Host — where harnesses actually live

- **Dedicated process**: Runs harnesses independently of the windows that display their sessions.
- **Keeps working:** Sessions carry on in the background while you keep coding, and survive a window reload.
- **Synced across windows**: One session can render in several VS Code windows at once and stay in sync.
- **Local or remote:** A harness on the Agent Host can also run on a remote machine over SSH or a dev tunnel.



> *Copilot, Claude and Codex sessions run on the Agent Host — enable it with chat.agentHost.enabled, which your organisation may control. Every chat in one session shares the same folder or worktree.*

COMPARE

Local vs Agent Host sessions — key differences

Feature	Local	Agent Host (Copilot)
Runs in	VS Code extension host	Separate Agent Host process
Blocks editor?	Yes — foreground	No — keep coding
Parallel sessions	One active	Many simultaneous
Built-in tools	Full access	Built-in + MCP via harness SDK
Extension tools	Full access	Not available
MCP servers	All configured	Configured MCP servers
Isolation	Your workspace, edited in place	Folder or Worktree
Survives restart	No — tied to editor	Yes — independent process

Short, interactive, tool-heavy work stays local; long-running, parallel, or risky work goes to the Agent Host.

Choosing the right harness — decision guide

Need the debugger or extension tools?	**Local**
Want to keep coding while it runs?	**Copilot**
Need several agents in parallel?	**Copilot + New Worktree**
Result should be a Pull Request?	**Cloud**
Want Anthropic or OpenAI behaviour?	**Claude or Codex**

Folder vs Worktree isolation — where the agent works

Aspect	Folder isolation	Worktree isolation
Changes land in	Your workspace, in place	A separate Git worktree folder
How to choose	Leave New Worktree unchecked	Tick New Worktree in the picker
Needs Git?	No	Yes — a Git repository
Permission level	Default · Assisted · Bypass	Bypass Approvals — fixed
Committing	You commit as usual	Auto-commit each turn
Parallel agents	Risk of file conflicts	Safe — isolated copies
Review flow	Diff in your working tree	Review, then merge the worktree
Harness support	Every harness	Copilot · Claude · Codex

The Agent Window — a dedicated space for agent sessions

What the Agent Window gives you

- Dedicated surface — agent sessions run in their own window, separate from inline chat.
- Non-blocking — the agent keeps working while you continue editing in the editor.
- Parallel sessions — start several agents at once and switch between them from one list.
- Live progress — see planning, tool calls, file edits and terminal output as they happen.
- Inline review — approve or reject proposed changes, or open the diff before keeping them.
- Session continuity — a session survives editor reloads, so long runs can be reattached.
- Handoff ready — promote a local session to Agent Host or Cloud for a pull request.

Three tool types extend every Copilot agent

Built-in

Pre-installed in VS Code. Always available.

- #editFiles
- #runCommands
- #codebase / #usages
- #problems
- #fetch · #findSymbol

MCP

External via Model Context Protocol.

- GitHub MCP Server
- Figma · Postman · Terraform
- Database tools
- Custom APIs
- BYOR registry (Apr 2026)

Extension

Contributed by VS Code / VS / JetBrains extensions.

- Test runners (@test)
- Debuggers (@debug)
- Linters
- Language servers
- IDE-specific commands

Built-in tools — core capabilities every agent inherits

Editing

#editFiles

Create, modify, or delete files in workspace.

Terminal

#runCommands

Run shell commands — install, build, test.

Search

#codebase

Semantic search across the indexed project.

Search

#changes

Find recent edits / diffs since last commit.

Search

#usages

Find references and call sites of a symbol.

Diag

#problems

Read compiler / lint diagnostics from the IDE.

Web

#fetch

Pull live content from a URL into context.

Nav

#findSymbol

Language-aware symbol navigation (C#, TS, …).

MCP tools — connect agents to your external systems

Why MCP?

- One open standard for connecting any AI to any tool/data.
- Replaces dozens of bespoke integrations — build once.
- Governance: BYOR registry allowlist (Apr 2026).
- Linux Foundation AAIF stewardship since Dec 2025.

mcp.json

```
// .vscode/mcp.json
{
  "servers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/"
    }
  }
}
```

Popular MCP servers and what they unlock

GitHub	Repos, issues, PRs, commits, code search, Coding Agent dispatch	**Figma**	Design-to-code via Dev Mode MCP
Postman	Run, document, and test APIs from the IDE	**Database**	Postgres / SQL Server / Mongo grounding for queries
Sentry	Pull live errors + breadcrumbs into the debug loop	**Azure DevOps**	Work items, pipelines, wiki content

How tools are used in prompts — pickers, mentions, tool sets

Enabling tools

- Tools picker (⚙) in the Chat view.
- Type # in chat to see available tools.
- Enable / disable per-request or per-session.
- MCP tools are opt-in (off by default).

prompts

```
// In Chat
#fetch "https://api.example.com/data"
#codebase find the auth middleware
#changes show recent modifications
#problems list current errors

// In agent mode the agent picks tools
// automatically based on the goal
```

Tool sets — bundle related tools and call them with one mention

settings.json · toolset

```
// settings.json
"reader": {
  "tools": ["search/changes", "search/codebase", "read/problems", "search/usages"],
  "description": "Tools for reading and gathering context",
  "icon": "book"
}
// Use in prompt:  #reader analyse the auth flow
```

Local agent modes — pick the right one for the task

Agent

Autonomous multi-step execution. Edits files, runs commands, iterates on errors.

When

Full implementation, multi-file change.

Plan

Designs a detailed plan / blueprint first. You review and approve before code is written.

When

Big features needing design review.

Ask

Conversational Q&A. Reads but does NOT modify files.

When

Learning, exploration, quick answers.

Custom

Specialised personas via .agent.md — locked tools, scoped instructions.

When

Reviewer, planner, security-auditor, doc-writer.

Inline Agent Mode — agent capabilities inside inline chat. Works in VS Code and JetBrains.

Agent mode — autonomous pair programmer

What it can do

- Analyse the codebase and identify relevant files.
- Create and modify files across the project.
- Run terminal commands — install, build, test.
- Detect and auto-fix compile / lint errors.
- Monitor test output and iterate to green.
- Propose risky commands for approval (per permission level).
- Call MCP and extension tools as needed.

agent prompt

```
// Example prompt

Add user authentication with JWT.
Include login / register endpoints,
bcrypt password hashing, and a
middleware that gates /api/* routes.

// Agent will:
 1. Search codebase for existing auth
 2. Create auth controller + routes
 3. Install bcrypt, jsonwebtoken
 4. Create middleware files
 5. Run tests and fix any failures
 6. Summarise the diff for review
```

Plan mode — blueprint before build (/plan)

1. **Submit goal**
2. **Plan generated**
3. **Review & approve**
4. **Implement**

Plan handoff options

Start Implementation
Choose an implementation agent; it receives the plan and context. Best for small, well-scoped plans you want to drive interactively.

Hand off to a Copilot session
Continue on the Agent Host, optionally in a new worktree, so you can keep working while the agent runs.

Switch to Autopilot mode
Auto-approves every tool call and iterates until the task is done. Best for routine plans on areas covered by tests.

Ask mode & Custom agents — Q&A and specialised personas

Ask mode

Conversational Q&A. No file writes. Best for explaining code, architecture decisions, framework choice, and any "is this safe?" question.

custom agent

```
# .github/agents/reviewer.agent.md
---
name: Code Reviewer
description: Reviews code for quality
tools: ['search/codebase', 'read/problems']
---
You are a code reviewer. Focus on
code quality, security, and style.
```

Agent handoff — Plan → Copilot → Cloud

1

Plan (local)

/plan designs the implementation. You review and approve before any code is written.

2

Copilot (Agent Host)

Hand off from the Session Target dropdown. Tick New Worktree so the agent works in an isolated copy.

3

Cloud (PR)

/delegate to the cloud agent. Pushes a branch, runs CI, self-fixes, opens a draft PR with self-review.

4

Review & merge

You review the diff. @copilot in comments to iterate. Merge when ready.

Conversation history carries across the handoff — the Copilot session sees the plan; the cloud agent sees what the session did. Use /remote on to steer a running session from GitHub.com or GitHub Mobile.

Agent handoff — in practice

Step 1 · Plan

```
// Step 1 — /plan prompt

Add unit tests for the auth module
covering login, register and the
password-reset flows.

// The plan agent returns:
//   - files to create / modify
//   - test cases per flow
```

Step 2 · Copilot

```
// Step 2 — hand off to Copilot

Session Target dropdown
  -> Copilot
     [x] New Worktree  (isolated)
     [ ] New Worktree  (in place)

// History carries over. Runs on
// the Agent Host while you code.
```

Step 3 · Cloud

```
// Step 3 — /delegate to the cloud agent (from the Copilot session)

/delegate Create a PR with these test changes.
         Run the test suite in CI and fix any failures before requesting review.

// Cloud agent will:  create branch -> push -> run CI -> self-fix -> open draft PR
// You review the diff and merge.   @copilot in PR comments to iterate.
```

Custom Agents — personas with scoped tools and pinned models

Configure Copilot to adopt a specific role — Security Reviewer, Solution Architect, Planner, Code Reviewer — and switch between them with one click.

Instructions

What to do, what to avoid — written in plain Markdown.

Tools

What it can access or modify — scoped per role.

Model preference

Which LLM to use, with fallback options.

Handoffs

Suggested next agents at the end of a turn.

Three sharing levels: workspace (.github/agents/) — shared with the team via Git · user profile — personal, follows you across repos · .claude/agents/ — Claude Code compatible.

The .agent.md file — Planner example

planner.agent.md

```
---
name: Planner
description: Generate plans for new features
tools: ['fetch','githubRepo','search','usages']
model: ['Claude Sonnet 5.5','GPT-5.3-Codex']
handoffs:
  - label: Implement Plan
    agent: agent
    send: false
---

# Instructions
You are in planning mode.

Do NOT make any code edits.

For every request:
 1. Identify the affected modules.
 2. Enumerate at least two options.
 3. Recommend one and justify it.
```

What this enforces

Read-only tools: no editFiles, so it cannot modify code during planning.

Model fallback: tries Claude first, falls back to Codex on rate limits.

Handoff button: "Implement Plan" appears at end of turn → switches to agent mode.

send: false: user previews the handoff prompt before it fires.

Result: safe planning by design — not by discipline.

Handoffs — chain agents into a guided workflow

Planner

Read-only. Designs the approach.

Coder

Edits code. Implements the approved plan.

Reviewer

Read-only. Critiques the diff for safety + style.

Done

You merge; agent goes home.

send: false — user previews

The handoff button surfaces a pre-filled prompt for the next agent. The user reads it, edits if needed, then fires. Good for steps where intent must be confirmed (anything that edits code).

send: true — auto-submits

Useful for trusted, deterministic transitions — e.g. "Lint Docs" after the Doc-Writer agent finishes. Combine with org policy to forbid send:true on agents with editFiles permissions.

Frontmatter reference and tool priority

Field	Purpose
`name`	Display name in dropdown
`description`	Placeholder shown to the user
`tools`	Array of tool / MCP names; e.g. ['codebase','search']
`model`	LLM with fallback; e.g. ['Claude Opus 5.5','GPT-5.6 Sol']
`handoffs`	Next-step suggestions (label, agent, prompt, send)
`user-invokable`	false hides the agent from the dropdown (subagents)
`agents`	Which subagents can be used

Tool priority

Highest → lowest:

- ① **Tools in the prompt file**
- ② Tools from referenced custom agent
- ③ Default tools of the selected agent

Design principles

- Least privilege — only the tools needed.
- Single responsibility — one agent, one job.
- Handoffs guide; do not automate blindly.

Key takeaways

1

Agents = autonomous loop

Plan → tool → execute → evaluate → iterate. The model picks tools; you set the scope.

2

Three tool types

Built-in core, MCP for external systems, Extensions for IDE features. Max 128 per request.

3

Four local modes

Agent (do), Plan (design first), Ask (Q&A), Custom (.agent.md personas with scoped tools).

4

Five harnesses

Local · Copilot · Claude · Codex · Cloud. Copilot, Claude and Codex run on the Agent Host.

5

Permissions and Autopilot

Default, Assisted (LLM judge), Bypass. Autopilot is an agent mode, not a level. Org policy controls availability.

6

Plan → Copilot → Cloud

Design with /plan, execute in a worktree on the Agent Host, ship through the cloud agent. Best of all three.

Thank you.

Questions, war stories, and agent diaries welcome.

Sandeep Soni · Deccansoft Software Services Pvt Ltd